



Pizza Express

Sistema de Gestión de Pedidos e Inventario

Plan de pruebas del sistema

Proyecto de Ingeniería de Software
Católica del Norte Fundación Universitaria



- **Proyecto:** Pizza Express — Sistema de Gestión de Pedidos e Inventario
- **Versión del sistema:** 1.0 (MVP)
- **Documento:** Plan de pruebas

1. Objetivo

Comprobar que Pizza Express cumple los requerimientos funcionales definidos para la versión 1, con especial atención a las dos reglas críticas del negocio: que el inventario se descuente **automáticamente** al vender una pizza y que la venta se **bloquee** cuando falta un insumo.

2. Alcance

Qué se prueba

- Registro de pedidos en las tres modalidades (mesa, para llevar y domicilio).
- Descuento automático del inventario según la receta de cada pizza y el tamaño.
- Bloqueo de la venta y mensajes de error cuando no hay inventario suficiente.
- Alertas de stock mínimo y reabastecimiento.
- Ciclo de estados del pedido y devolución de inventario al cancelar.
- Reportes de ventas y su coherencia con la base de datos.
- Control de acceso por rol.
- Presencia de la marca (nombre y logo) en todas las pantallas.

Qué no se prueba

Queda fuera lo que la versión 1 no implementa: pagos en línea, seguimiento GPS de repartidores y programa de fidelización. Tampoco se realizan pruebas de carga, de estrés ni de seguridad avanzada, por tratarse de un prototipo académico de un solo puesto de trabajo.

3. Entorno de pruebas

Elemento	Valor
Sistema operativo	macOS (Darwin 25.6)
Lenguaje	Python 3.13
Framework web	Flask 3.1
Base de datos	SQLite (módulo <code>sqlite3</code> de la biblioteca estándar)
Navegador	Google Chrome
Datos de partida	Base recién sembrada: 18 ingredientes, 6 pizzas, 3 usuarios y 2 pedidos de ejemplo

Para reproducir el entorno exacto basta con borrar `datos/pizzaexpress.db` y volver a ejecutar `python ejecutar.py`: la aplicación recrea la base y la vuelve a sembrar con los mismos datos.

Datos de partida relevantes

La semilla deja a propósito varios ingredientes en alerta, para poder probar las reglas sin preparar nada:

Ingrediente	Stock tras la semilla	Stock mínimo	Situación
Queso azul	0 kg	0,5 kg	Agotado — bloquea la pizza Cuatro Quesos
Aceitunas negras	0,8 kg	1 kg	Por acabarse
Champiñones	1,5 kg	2 kg	Por acabarse
Jamón	2,3 kg	3 kg	Por acabarse
Queso mozzarella	4,85 kg	8 kg	Por acabarse

4. Estrategia de pruebas

Se trabajó en tres niveles:

Nivel	Qué cubre	Cómo se ejecuta
Unitario	Reglas de negocio de <code>app/servicios/</code> de forma aislada	<code>unittest</code> sobre una base en memoria (18 pruebas automáticas)
Integración	Vistas + servicios + base de datos, a través de peticiones HTTP reales	Peticiones al servidor con sesión iniciada, comprobando después el estado de la base de datos
Sistema / aceptación	Flujos completos de cada rol, de principio a fin	Recorrido manual y automatizado de los casos CP-01 a CP-17

Además se aplicaron dos técnicas de diseño de casos:

- **Partición de equivalencia:** pedidos válidos frente a pedidos inválidos (sin pizzas, sin dirección, con cantidad cero).
- **Valores límite:** pedir justo la cantidad disponible de un insumo frente a pedir una unidad más de la disponible.

Criterios de entrada

- La aplicación arranca sin errores y crea la base de datos.
- Las 18 pruebas unitarias pasan.

Criterios de salida (aceptación)

- El 100 % de los casos de prueba de los requerimientos RF1 a RF7 termina en estado **Correcto**.
 - No queda ningún defecto abierto de severidad alta.
 - Las 12 pantallas muestran el nombre «Pizza Express» y el logo.
-

5. Casos de prueba

Todos los casos se ejecutaron sobre una base recién sembrada. La columna «Resultado obtenido» recoge la salida real de la última ejecución.

5.1 Acceso y seguridad

ID	RF	Precondición	Pasos	Resultado esperado	Resultado obtenido	Estado
CP-01	—	Base sembrada	Entrar en <code>/login</code> con <code>cajero / cajero123</code>	Inicia sesión y va al panel	Redirige a <code>/</code>	Correcto
CP-02	—	Base sembrada	Entrar con <code>admin</code> y una clave equivocada	Mensaje de error y no entra	«Usuario o contraseña incorrectos.»	Correcto
CP-03	—	Sin sesión	Abrir <code>/</code> directamente	Redirige a la pantalla de acceso	Termina en <code>/login</code>	Correcto
CP-12	—	Sesión de cajero	Abrir <code>/reportes/</code>	Se rechaza el acceso	«Tu rol (Cajero / mesero) no tiene permiso para esa sección.»	Correcto
CP-13	—	Sesión de pizzero	Abrir <code>/pedidos/</code>	Se rechaza el acceso	Redirige al panel	Correcto

5.2 Toma de pedidos y descuento de inventario

ID	RF	Precondición	Pasos	Resultado esperado	Resultado obtenido	Estado
CP-04	RF1, RF2	Sesión de cajero. Mozzarella 4,85 kg; harina 23,62 kg	Pedido en mesa : 1 Margarita mediana (receta: 0,25 kg de mozzarella y 0,30 kg de harina)	Se crea el pedido y el stock baja exactamente lo que dice la receta	Pedido creado. Mozzarella 4,85 → 4,60 kg ; harina 23,62 → 23,32 kg	Correcto
CP-05	RF1, RF2	Sesión de cajero. Pepperoni 3,88 kg	Pedido para llevar : 1 Pepperoni familiar (0,12 kg × factor 1,6 = 0,192 kg)	El tamaño familiar multiplica el consumo por 1,6	Pepperoni 3,88 → 3,688 kg	Correcto
CP-06	RF1	Sesión de cajero	Pedido a domicilio : 2 Hawaianas medianas, con dirección	Se crea el pedido	Pedido creado (<code>/pedidos/5</code>)	Correcto
CP-08	RF1	Sesión de cajero	Pedido a domicilio dejando la dirección vacía	Se rechaza indicando el dato que falta	«Falta el dato «Dirección de entrega» para un pedido a domicilio.»	Correcto
CP-09	RF1	Sesión de cajero	Guardar un pedido sin poner cantidad a ninguna pizza	Se rechaza	«Agrega al menos una pizza al pedido.»	Correcto

5.3 Bloqueo de venta por falta de insumos (regla crítica)

ID	RF	Precondición	Pasos	Resultado esperado	Resultado obtenido	Estado
CP-07	RF5	Queso azul agotado (0 kg). Parmesano 2 kg, cheddar 2,5 kg	Intentar vender 1 Cuatro Quesos familiar	La venta se bloquea, se indica el insumo que falta y no se modifica ningún ingrediente	«No hay inventario suficiente para este pedido. Falta «Queso azul»: se necesitan 0.08 kg y solo hay 0 kg.» Parmesano sigue en 2,0 kg y cheddar en 2,5 kg	Correcto
CP-15	RF3, RF5, RF6	Situación anterior	Como administrador, reabastecer 5 kg de queso azul y repetir el pedido	Tras el reabastecimiento la pizza sí se puede vender	Queso azul → 5,0 kg. Pedido creado (/pedidos/6)	Correcto

Este par de casos es el más importante del plan: demuestra que el sistema **no vende lo que no puede preparar** y que la mitigación del riesgo «quiebre de un ingrediente en pleno servicio» funciona de punta a punta.

5.4 Ciclo de vida del pedido

ID	RF	Precondición	Pasos	Resultado esperado	Resultado obtenido	Estado
CP-10	RF7	Pedido PED-0003 pendiente. Sesión de pizzero	Avanzar el pedido por el tablero: en preparación → lista → entregado	El pedido recorre los tres estados	Estado final en la base de datos: entregado	Correcto
CP-11	RF2, RF7	Pedido PED-0004 (Pepperoni familiar) activo. Pepperoni 3,688 kg	Cancelar el pedido	El inventario vuelve a su valor anterior	Pepperoni 3,688 → 3,88 kg (devuelve los 0,192 kg)	Correcto

5.5 Alertas y reportes

ID	RF	Precondición	Pasos	Resultado esperado	Resultado obtenido	Estado
CP-14	RF3	Base sembrada	Abrir <code>/inventario/alertas</code>	Se listan los ingredientes en alerta, separando agotados de los que están por acabarse	«Agotados (1)» y «Por acabarse (4)»	Correcto
CP-16	RF4	Varios pedidos registrados, uno de ellos cancelado	Abrir el reporte de ventas y comparar con una consulta directa a la base de datos	Las cifras de la pantalla coinciden con la base de datos y el pedido cancelado no cuenta	Pantalla: ventas \$ 299.600, 5 pedidos, 8 pizzas, ticket \$ 59.920. Base de datos: 299.600, 5, 8, 59.920	Correcto

5.6 Marca (requisito de imagen corporativa)

ID	Precondición	Pasos	Resultado esperado	Resultado obtenido	Estado
CP-17	Sesión de administrador	Recorrer las 12 pantallas y comprobar que en el HTML aparecen el texto «Pizza Express» y la imagen del logo	Las 12 pantallas muestran nombre y logo; el archivo del logo se descarga correctamente	12/12 pantallas correctas. <code>/assets/logo.png</code> : 5.586 bytes, cabecera PNG válida	Correcto

Pantallas verificadas: acceso, panel, lista de pedidos, nuevo pedido, detalle de pedido, cocina, menú, inventario, alertas, nuevo ingrediente, editar ingrediente y reportes.

6. Pruebas automatizadas

El archivo `pruebas/test_servicios.py` contiene **18 pruebas unitarias** sobre la capa de servicios. Se ejecutan con:

```
python -m unittest discover -s pruebas -v
```

Cada prueba trabaja sobre una base de datos **en memoria**, creada y sembrada de nuevo antes de cada caso, por lo que no modifican los datos de la aplicación y pueden ejecutarse tantas veces como se quiera.

Clase de pruebas	Nº	Qué comprueba
PruebasDeInventario	5	Que el consumo suma los ingredientes compartidos entre pizzas; que el tamaño multiplica el consumo; que las alertas separan agotados de bajos sin solaparse; que el reabastecimiento suma al stock y que rechaza cantidades negativas
PruebasDePedidos	10	Descuento al crear el pedido; total según el tamaño; bloqueo por insumo faltante; que una venta bloqueada no toca el inventario; obligatoriedad de la dirección; rechazo de pedidos vacíos; ciclo de estados en orden; imposibilidad de saltar estados; devolución de stock al cancelar; imposibilidad de cancelar un pedido entregado
PruebasDeReportes	1	Que un pedido cancelado no cuenta como venta
PruebasDeAutenticacion	2	Acceso con credenciales correctas e incorrectas

Resultado de la última ejecución: Ran 18 tests – OK (0 fallos, 0 errores).

7. Matriz de trazabilidad

Relaciona cada requerimiento con los casos que lo verifican, para demostrar que no queda ningún requerimiento sin probar.

Requerimiento	Casos de sistema	Pruebas automáticas
RF1 — Pedidos en mesa, para llevar y domicilio	CP-04, CP-05, CP-06, CP-08, CP-09	test_el_domicilio_exige_direccion , test_un_pedido_sin_pizzas_se_rechaza
RF2 — Descuento automático por pizza	CP-04, CP-05, CP-11	test_crear_un_pedido_descuenta_el_inventario , test_el_tamano_multiplica_el_consumo , test_el_consumo_suma_los_ingredientes_compartidos , test_cancelar_devuelve_los_ingredientes
RF3 — Alertas de stock mínimo	CP-14, CP-15	test_las_alertas_separan_agotados_de_bajos
RF4 — Reportes básicos de ventas	CP-16	test_un_pedido_cancelado_no_cuenta_como_venta
RF5 — Bloqueo de venta por insumo faltante	CP-07, CP-15	test_se_bloquea_la_venta_si_falta_un_insumo , test_una_venta_bloqueada_no_toca_el_inventario
RF6 — Gestión de inventario y precios	CP-15	test_reabastecer_suma_al_stock , test_no_se_puede_reabastecer_una_cantidad_negativa
RF7 — Seguimiento del pedido en cocina	CP-10, CP-11	test_el_ciclo_de_estados_avanza_en_orden , test_no_se_puede_salvar_un_estado , test_un_pedido_entregado_ya_no_se_puede_cancelar
Control de acceso por rol	CP-01, CP-02, CP-03, CP-12, CP-13	test_credenciales_correctas , test_credenciales_incorrectas
Marca en todas las pantallas	CP-17	—

8. Defectos encontrados y corregidos

Durante las pruebas se detectaron dos defectos de presentación, ambos corregidos y verificados:

ID	Descripción	Severidad	Corrección	Estado
DEF-01	En las tablas estrechas, los precios se partían en dos líneas («\$ 60.» / «800»), lo que dificultaba leer el total del pedido	Baja	Se añadió <code>white-space: nowrap</code> a la clase <code>.derecha</code> en <code>assets/estilos.css</code>	Cerrado
DEF-02	El texto de ayuda del formulario de pedido («Obligatorio para pedidos en mesa y a domicilio») aparecía en la misma línea del campo en vez de debajo	Baja	Se añadió la regla <code>small.ayuda { display: block; }</code> en <code>assets/estilos.css</code>	Cerrado

No se encontraron defectos de severidad media ni alta en la lógica de negocio.

9. Conclusiones

- Los **17 casos de prueba de sistema** y las **18 pruebas automáticas** terminaron en estado correcto, por lo que se cumplen los criterios de salida.
- Las dos reglas críticas del proyecto quedaron demostradas con datos concretos: el descuento automático coincide al detalle con las recetas (CP-04 y CP-05) y la venta se bloquea sin tocar el inventario cuando falta un insumo (CP-07).
- Los requerimientos RF1 a RF7 están cubiertos por al menos un caso de prueba, tal como muestra la matriz de trazabilidad.
- Los únicos defectos hallados fueron de presentación, de severidad baja, y quedaron cerrados.

Recomendaciones para próximas versiones

1. Automatizar los casos de sistema con una herramienta de pruebas de interfaz, para no tener que repetirlos a mano en cada entrega.
2. Añadir pruebas de concurrencia: si dos cajeros registran pedidos al mismo tiempo sobre el último insumo disponible, conviene verificar el comportamiento del bloqueo.
3. Incluir pruebas de usabilidad con usuarios reales de la pizzería (cajero y pizzero).